Guía paso a paso — Clase 1: Introducción a las Redes Neuronales

Temas: presentación del curso · instalación de herramientas · inteligencia artificial y redes neuronales · aprendizaje automático
Duración: ~2 h 15 min
Material: diapositivas `Clase1_Diapositivas` (PDF y PowerPoint, 19 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales

Esta guía explica **todo** lo que aparece en las diapositivas desde cero, como si nunca hubieras visto el tema. Tiene tres partes:

- **Parte A:** preparar las computadoras **antes** de clase. Es una instalación rigurosa, con una verificación en cada paso.
- **Parte B:** qué explicar en cada diapositiva, con definiciones sencillas y ejemplos.
- **Parte C:** problemas frecuentes y cómo resolverlos.

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar qué es la inteligencia artificial y dónde encajan las redes neuronales.
2. Distinguir las tareas del aprendizaje automático (regresión, clasificación, agrupamiento…) y sus tres tipos (supervisado, no supervisado, por refuerzo).
3. Tener su computadora lista para hacer los ejercicios del curso.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación y motivación	1–5
0:20	Herramientas e instalación	6–8
0:55	Descanso	—
1:05	Inteligencia artificial y redes neuronales	9–12
1:35	El aprendizaje automático	13–17
2:05	Quiz y cierre	18–19

PARTE A — Preparar las computadoras (hazlo antes de clase)

A.0 Conceptos mínimos antes de instalar

Palabra	Qué es, en simple
Python	El lenguaje de programación que usaremos.
Paquete / librería	Código que otros ya escribieron y que instalamos para usarlo. Ejemplo: **NumPy** sirve para hacer cuentas con vectores y matrices.
Anaconda / conda	Programa que instala Python y administra "ambientes".
Ambiente	Una "caja" aislada con su propia versión de Python y sus paquetes. Así un proyecto no rompe a otro. El nuestro se llamará `nni` (*neural network introduction*).
pip	El instalador de paquetes de Python.
Terminal	Ventana donde se escriben comandos de texto. Usaremos **Anaconda PowerShell Prompt**.
Git	Programa para descargar y versionar código.
GitHub	Sitio web donde se guarda código. Allí está el repositorio del curso.
Repositorio (repo)	Carpeta de un proyecto con todos sus archivos.
Clonar	Descargar un repositorio a tu computadora con Git.
Notebook (.ipynb)	Documento que mezcla texto y celdas de código que se ejecutan una por una. Los ejercicios vienen así.
Jupyter	El programa que ejecuta los notebooks.
VS Code	Editor de código donde abriremos los notebooks.
Kernel	El "motor" de Python que ejecuta las celdas de un notebook. Debemos elegir el de nuestro ambiente `nni`.

Regla de oro: después de cada paso hay una **verificación**. No avances si la verificación falla; ve a la Parte C.

A.1 Elegir el camino

Camino	Cuándo usarlo	Qué necesita
Camino 1: Google Colab	Computadoras sin permisos para instalar, poco tiempo o plan B	Navegador y cuenta de Google
Camino 2: Instalación local	Es lo recomendado en el curso: da experiencia real de desarrollo	Instalar Anaconda, Git y VS Code (unos 40 minutos y unos 5 GB libres)

Recomendación: prepara el **Camino 2** en tu computadora y ten listo el **Camino 1** como respaldo para los alumnos que tengan problemas.

A.2 Camino 1 — Google Colab (sin instalar nada)

1. Entra a https://colab.research.google.com e inicia sesión con tu cuenta de Google.
2. Menú **Archivo** → **Abrir cuaderno** → pestaña **GitHub**.
3. Pega `datahuba/curso-redes-neuronales` y presiona Enter.
4. Elige el notebook, por ejemplo `notebooks/1_01_mcculloch_pitts.ipynb`.
5. Haz clic en la primera celda de código y presiona **Shift + Enter** para ejecutarla.
6. Para guardar tus cambios: **Archivo** → **Guardar una copia en Drive**. Si no lo haces, se pierden.

Verificación: crea una celda nueva (botón **+ Código**), escribe esto y ejecútala:

```
import numpy as np
print(np.__version__)
```

Si aparece un número de versión (por ejemplo `1.26.4`), Colab funciona.

Notebooks que usan la carpeta `nni` o archivos de datos (como `1_03_2`): en Colab necesitan el repositorio completo. Ejecuta esto en la primera celda:

```
!git clone https://github.com/datahuba/curso-redes-neuronales
%cd curso-redes-neuronales/notebooks
```

A.3 Camino 2 — Instalación local en Windows

Son **7 pasos**, en este orden:

- Pasos 1 a 3: instalar tres programas, con clics.
- Pasos 4 a 7: escribir comandos en **una sola** terminal, Anaconda PowerShell Prompt.

Paso 1. Instalar Anaconda

1. Entra a https://www.anaconda.com/download
2. Descarga el instalador para **Windows** (64 bits).
3. Ejecuta el archivo descargado (`Anaconda3-...-Windows-x86_64.exe`).
4. En el instalador:
   - **Next** → **I Agree**.
   - Elige **Just Me (recommended)**.
   - Deja la carpeta que propone. Es importante que la ruta no tenga espacios ni acentos; normalmente es `C:\Users\TuUsuario\anaconda3`.
   - En opciones avanzadas, deja marcadas las que vienen por defecto. **No** marques "Add Anaconda to my PATH".
   - **Install** y espera de 5 a 15 minutos → **Finish**.

Verificación:

1. Abre el menú Inicio, escribe `Anaconda PowerShell Prompt` y ábrelo.
2. Debe verse una línea que empieza con `(base)`, por ejemplo:

```
(base) PS C:\Users\TuUsuario>
```

3. Escribe y presiona Enter:

```
conda --version
```

Debe responder algo como `conda 24.x.x`.

> Anaconda tiene una interfaz gráfica (Anaconda Navigator), pero es lenta. Usaremos la terminal.

Paso 2. Instalar Git for Windows

1. Entra a https://git-scm.com/download/win
2. Descarga **64-bit Git for Windows Setup** y ejecútalo.
3. Acepta todas las opciones por defecto (**Next** en todo) → **Install** → **Finish**.

Verificación: cierra y vuelve a abrir Anaconda PowerShell Prompt (así reconoce el programa nuevo) y escribe:

```
git --version
```

Debe responder algo como `git version 2.4x.x.windows.1`.

Paso 3. Instalar VS Code y sus extensiones

1. Entra a https://code.visualstudio.com, descarga e instala con las opciones por defecto. Marca **"Agregar a PATH"** si aparece.
2. Abre VS Code.
3. Instala dos extensiones:
    1. Haz clic en el ícono de **Extensiones** de la barra izquierda (cuatro cuadritos) o presiona `Ctrl + Shift + X`.
    2. Busca **Python** (de Microsoft) → **Install**.
    3. Busca **Jupyter** (de Microsoft) → **Install**.

Verificación: en la barra izquierda, dentro de Extensiones, aparecen **Python** y **Jupyter** como instaladas.

Paso 4. Crear y activar el ambiente `nni`

En **Anaconda PowerShell Prompt** escribe:

```
conda create --name nni python=3.11
```

Qué significa:

- `conda create` → crea un ambiente nuevo.
- `--name nni` → le pone de nombre `nni`.
- `python=3.11` → le instala Python 3.11. En el video se usa `python=3`; 3.11 es una versión 3 concreta y estable.

Cuando pregunte `Proceed ([y]/n)?`, escribe `y` y presiona Enter. Tarda de 1 a 3 minutos.

Luego actívalo:

```
conda activate nni
```

Verificación: el inicio de la línea debe cambiar de `(base)` a `(nni)`:

```
(nni) PS C:\Users\TuUsuario>
```

⚠️ **Cada vez que abras una terminal nueva**, el ambiente vuelve a `(base)`. Debes repetir `conda activate nni`.

Paso 5. Descargar (clonar) el repositorio del curso

En la misma terminal, con `(nni)` visible:

```
cd ~
```

```
mkdir projects
```

```
cd projects
```

```
git clone https://github.com/datahuba/curso-redes-neuronales.git
```

```
cd curso-redes-neuronales
```

Qué significa:

- `cd ~` → ir a tu carpeta de usuario.
- `mkdir projects` → crear una carpeta `projects`. Si ya existía, muestra un error en rojo; no pasa nada, continúa.
- `cd projects` → entrar a esa carpeta.
- `git clone <url>` → descargar una copia del repositorio.
- `cd curso-redes-neuronales` → entrar a la carpeta del curso.

Verificación: escribe `ls`. Deben aparecer `docs`, `notebooks`, `README.md` y `requirements.txt`.

> **Alternativa sin Git:** en GitHub, botón verde **Code** → **Download ZIP**. Descomprímelo en `C:\Users\TuUsuario\projects` y entra a esa carpeta con `cd`.
>
> En el video se usa la terminal **Git Bash** para clonar; también funciona, pero así usamos una sola ventana para todo.

Paso 6. Instalar los paquetes

Dentro de la carpeta `curso-redes-neuronales`, con `(nni)` activo:

```
pip install -r requirements.txt
```

`-r requirements.txt` le dice a pip que instale **la lista de paquetes del curso**, que viene en ese archivo:

Paquete	Para qué
`jupyter`	Ejecutar notebooks
`numpy`	Vectores, matrices y cuentas
`pandas`	Tablas de datos
`matplotlib`	Gráficas
`scikit-learn`	Utilidades de aprendizaje automático (el notebook `1_03_2` lo necesita)
`pillow`	Abrir imágenes
`ipykernel`	Que VS Code reconozca el ambiente como kernel

Tarda de 2 a 5 minutos. La última línea debe decir `Successfully installed ...`.

Verificación:

```
python -c "import numpy, pandas, matplotlib, sklearn, PIL; print('TODO OK')"
```

Debe imprimir `TODO OK`. Si dice `ModuleNotFoundError`, revisa que estés en `(nni)` y repite el Paso 6.

Paso 7. Abrir los notebooks en VS Code (verificación final)

1. En VS Code: **File** → **Open Folder...** (Archivo → Abrir carpeta) → elige `C:\Users\TuUsuario\projects\curso-redes-neuronales` → **Seleccionar carpeta**. Si pregunta si confías en los autores, elige **Yes, I trust**.
2. En el panel izquierdo abre `notebooks` → `1_01_mcculloch_pitts.ipynb`.
3. **Elige el kernel (muy importante):**
    1. Arriba a la derecha, haz clic en **Select Kernel**.
    2. Elige **Python Environments...**.
    3. Elige el que diga **nni**, por ejemplo `nni (Python 3.11.x)`.
4. Crea una celda (botón **+ Code**), escribe esto y ejecútalo con **Shift + Enter**:

```
import sys, numpy as np
print(sys.executable)
print(np.__version__)
```

Verificación final:

- La primera línea debe contener `envs\nni`; eso prueba que usas el ambiente correcto.
- La segunda debe ser un número de versión.

✅ Si ves ambas cosas, **la computadora está lista**. Borra la celda de prueba.

Resumen de comandos (para copiar)

Todo en **Anaconda PowerShell Prompt**:

```
conda create --name nni python=3.11
```

```
conda activate nni
```

```
cd ~
```

```
mkdir projects
```

```
cd projects
```

```
git clone https://github.com/datahuba/curso-redes-neuronales.git
```

```
cd curso-redes-neuronales
```

```
pip install -r requirements.txt
```

Alternativa sin VS Code: con `(nni)` activo, dentro de `curso-redes-neuronales`:

```
jupyter notebook
```

Se abre el navegador con la lista de archivos. Para cerrarlo, en la terminal presiona `Ctrl + C`.

A.4 Checklist del día de la clase

- [ ] Tu computadora pasó la **verificación final** del Paso 7.
- [ ] Probaste abrir un notebook en **Colab** (plan B).
- [ ] Tienes abiertas las diapositivas.
- [ ] Llegaste 15 minutos antes para probar el proyector.
- [ ] Si los alumnos instalarán en clase, avísales que traigan laptop con **5 GB libres** y permisos de administrador. Si pueden, que descarguen Anaconda en casa: son unos 900 MB.

PARTE B — Qué explicar en cada diapositiva

Junto a cada título va el tiempo aproximado.

Diapositiva 1 — Portada (2 min)

Qué decir: "Este es un curso introductorio. Vamos a entender cómo funcionan las redes neuronales **por dentro**: su teoría y cómo programarlas desde conceptos simples. Esto es la base para después estudiar aprendizaje profundo (*deep learning*) y construir cosas más complejas."

Diapositiva 2 — Agenda (1 min)

Hoy vemos 4 temas, uno por cada video del curso:

1. Presentación del curso.
2. Instalación de herramientas.
3. Inteligencia artificial y redes neuronales.
4. Aprendizaje automático.

Diapositiva 3 — Objetivos (1 min)

Lee los tres objetivos de la clase (ver arriba). Pide que al final de la clase ellos mismos digan si los cumplieron.

Diapositiva 4 — "Ya las usas todos los días" (10 min)

Actividad (5 min): pregunta "¿Qué aplicación con inteligencia artificial usaron hoy?". Anota las respuestas en el pizarrón.

Explica cada tarjeta:

- **Vehículos:** autos que ven la calle con cámaras y deciden cómo manejar.
- **Lenguaje:** Alexa, Google, traductores y chatbots entienden y producen texto.
- **Imagen:** los filtros de TikTok detectan tu cara y la modifican.
- **Salud:** detectan enfermedades en radiografías.
- **Videojuegos:** generan gráficos realistas más rápido. Calcular la luz real (*ray tracing*) es muy costoso.
- **Refuerzo:** programas que aprenden a jugar jugando muchas veces.
- **Generativa:** DALL·E y Stable Diffusion crean imágenes a partir de un texto. Hoy ya cuesta distinguir si una imagen la hizo una persona o una IA.
- **Robótica:** robots que perciben y se mueven.

Mensaje clave: las redes neuronales ya trabajan "por detrás" en muchos lados; por eso conviene saber cómo funcionan.

Diapositiva 5 — "¿Qué veremos en el curso?" (5 min)

Recorre los módulos:

1. **Introducción** (hoy).
2. **Principios:** la neurona de McCulloch y Pitts y el perceptrón.
3. **Red neuronal simple:** funciones de activación y descenso por gradiente.
4. **Redes multicapa:** retropropagación.
5. **Preliminares (opcional):** cálculo, álgebra lineal, probabilidad y Python. Quien sienta que le falta base, que empiece por aquí.

Cómo trabajamos cada tema: leer el libro → explicación en clase → ejercicio en papel → notebook en Python.

Diapositiva 6 — "Dos formas de hacer los ejercicios" (5 min)

- Los ejercicios son **notebooks** con instrucciones: solo hay que **rellenar partes del código**.
- Usamos solo **Python y NumPy**, "a pie", a propósito. Así entendemos qué pasa por dentro, en lugar de llamar a una función mágica como `.fit()` que lo hace todo sin que veamos nada.
- **Opción 1, Colab:** no se configura nada; funciona en el navegador.
- **Opción 2, tu máquina:** más trabajo al inicio, pero es como se trabaja en proyectos reales y en equipo.

Diapositiva 7 — "Instalación (1/2): tres programas" (10 min)

Muestra dónde se descarga cada programa (Pasos 1–3 de la Parte A) y para qué sirve:

- **Anaconda:** Python + ambientes.
- **Git:** descargar el código del curso.
- **VS Code:** editor; necesita las extensiones **Python** y **Jupyter**.

Si los alumnos ya los instalaron en casa, pasa directo a la siguiente.

Diapositiva 8 — "Instalación (2/2): los comandos" (20 min)

Haz la instalación en vivo (Pasos 4–7) y que los alumnos la repitan.

Comando	En palabras
`conda create --name nni python=3.11`	"Crea una caja llamada nni con Python dentro."
`conda activate nni`	"Entra a la caja. Fíjense que ahora dice (nni)."
`cd ~`, `mkdir projects`, `cd projects`	"Voy a mi carpeta de usuario, creo la carpeta de proyectos y entro."
`git clone <url>`	"Descarga una copia del repositorio del curso."
`cd curso-redes-neuronales`	"Entra a la carpeta del curso."
`pip install -r requirements.txt`	"Mete en la caja todas las herramientas de la lista."
VS Code → Abrir carpeta → kernel `nni`	"Abro el proyecto en el editor y elijo el motor correcto."

Termina cuando todos vean la **verificación final** del Paso 7. Quien no lo logre, que use Colab hoy y lo resuelva después con la Parte C.

Diapositiva 9 — "¿Qué es la inteligencia artificial?" (8 min)

- **No existe una definición única** aceptada por todos, ni un teorema que diga "esto sí es IA y esto no".
- El libro de **Russell y Norvig** (*Inteligencia Artificial: un enfoque moderno*) agrupa las definiciones en dos enfoques:
  1. **Imitar al humano:** construir máquinas que piensen como nosotros, duplicando lo que hace el cerebro. Problema: si copiamos perfectamente al humano, también copiamos sus errores.
  2. **Actuar racionalmente:** que la máquina **perciba** su entorno, **razone** (procese) y **actúe** para resolver el problema, aunque no lo haga como el cerebro.
- **Ejemplo:** una calculadora no piensa como un humano, pero resuelve sumas perfectamente. Es "racional" para esa tarea.

Diapositiva 10 — "El test de Turing" (10 min)

El problema: como no hay una definición de inteligencia, Alan Turing propuso una **prueba**.

Cómo funciona:

1. Una persona conversa por mensajes, como en un chat, con "alguien" que no ve.
2. Del otro lado puede haber una persona o una máquina.
3. Si la persona **no logra distinguir** si habla con una máquina, la máquina **pasa** el test y se considera inteligente.

¿Por qué es "operacional"? Porque solo mide si pasa o no; no le importa cómo funciona por dentro.

Qué necesita una máquina para pasarlo (explica las flechas en orden, como un flujo):

1. **Lenguaje natural:** recibir y entender el mensaje.
2. **Representación del conocimiento:** convertir el mensaje en algo que la máquina pueda manejar internamente.
3. **Inferencia:** a partir de ese conocimiento, generar una respuesta.
4. **Síntesis:** convertir la respuesta otra vez en lenguaje para el humano.
5. **Aprendizaje:** adaptarse a cada usuario distinto.

Conexión con el curso: con el aprendizaje profundo, las redes neuronales ya cubren todas esas partes:

- extraen características a una representación interna llamada **espacio latente**;
- hacen inferencia;
- generan la salida con un **decodificador** (texto o imagen);
- y sus parámetros internos (**pesos**) se ajustan a cada tarea.

Diapositiva 11 — "¿Qué tan lejos llegan las redes?" (8 min)

Dibuja **tres ejes** en el pizarrón:

Eje	Significa	Redes neuronales
Operacional	Resolver la tarea: controlar, jugar, reconocer, generar	**Muy buenas**: reconocen personas, manejan autos, crean imágenes
Razonamiento	Representar conocimiento, inferir como humano y, al extremo, tener conciencia	Limitadas, al menos por ahora
Explicabilidad	Poder garantizar cuándo funcionan y cuándo no, y por qué	Limitadas: a veces fallan y no sabemos exactamente por qué

Conclusión: son herramientas muy efectivas para resolver problemas, pero no tienen conciencia ni son completamente explicables. Por eso en el curso nos interesa mucho **cómo entrenarlas** bien.

Diapositiva 12 — "Dónde encajan las redes neuronales" (5 min)

Explica los recuadros, uno dentro de otro:

- **Inteligencia artificial** (el más grande): todo intento de que las máquinas hagan tareas inteligentes.
    - **Aprendizaje automático:** la parte de la IA que **aprende de ejemplos**.

- **Redes neuronales:** un tipo de aprendizaje automático formado por muchas unidades simples conectadas.
  - **Aprendizaje profundo:** redes con **muchas capas**; cada capa extrae rasgos más abstractos (bordes → formas → objetos).

Diapositiva 13 — "El aprendizaje automático" (8 min)

Definición (Arthur Samuel, 1959): "dar a las computadoras la capacidad de aprender sin ser programadas explícitamente".

Ejemplo del ajedrez:

- Queremos una computadora que gane al ajedrez.
- Un programador **no puede** escribir qué hacer en cada posición posible del tablero: hay muchísimas combinaciones, sería intratable.
- Solución: que la computadora **aprenda de la experiencia**, por ejemplo jugando muchas partidas.

Tres ingredientes (están en la diapositiva):

1. **Tarea:** ganar al ajedrez.
2. **Experiencia:** partidas jugadas o datos.
3. **Desempeño:** mejora después de observar; gana más seguido.

Diapositiva 14 — "Tarea 1: Regresión" (10 min)

Qué es: encontrar una **función** que relacione una **entrada** con una **salida numérica continua**. Ejemplo: entrada = metros cuadrados de una casa; salida = precio.

Explica la gráfica:

- **Puntos naranjas = datos** (la "experiencia" o *dataset*). Pueden venir de estudios, mediciones o simulaciones.
- **Línea azul = la función real.** En la vida real **no la conocemos**; solo tenemos los puntos.
- **Líneas grises = hipótesis:** funciones candidatas que proponemos para explicar los puntos.

Conceptos:

- Una hipótesis es **consistente** si explica bien los datos que tenemos.
- Pero eso no basta: queremos que también funcione con **datos futuros** del mismo fenómeno. Eso se llama **generalización**.

Pregunta a la clase: "¿Una curva que pase exactamente por todos los puntos haciendo zigzag es buena?" → No necesariamente: puede explicar estos datos y fallar con los nuevos.

Diapositiva 15 — "Otras tareas" (10 min)

1. **Clasificación:** asignar una **etiqueta** de una lista predefinida $\{l_1, \ldots, l_n\}$. Ejemplo: le das una foto y responde "perro" o "gato" (dataset *Dogs vs Cats* de Kaggle). A diferencia de la regresión, la salida es una **categoría**, no un número continuo.
2. **Agrupamiento (clustering): no hay etiquetas**; el algoritmo agrupa solo, por parecido. Ejemplo: con medidas como qué tan puntiagudas son las orejas o qué tan largos los bigotes, encuentra dos grupos

(que resultan ser perros y gatos). Cada grupo tiene su **media** (centro) y su **varianza** (qué tan dispersos están).

3. **Reducción de dimensionalidad:** describir algo con **menos características**. Ejemplo: en 2 ejes, perros y gatos están revueltos y ninguna recta los separa; si los proyectamos a 1 solo eje nuevo, basta un **umbral** (un punto) para separarlos.
4. **Modelado generativo:** aprender la **distribución** de los datos para **crear ejemplos nuevos**. Ejemplo: aprender "cómo es un rostro" y generar rostros que no existen.

Diapositiva 16 — "Tipos de aprendizaje" (10 min)

1. **Supervisado:**
   - La experiencia es una lista de **pares (x, y)**: x = el ejemplo (una figura), y = su etiqueta (cono, cubo, esfera).
   - El algoritmo busca la función que lleva de x a y. Ese proceso se llama **entrenamiento**.
   - Después debe acertar con ejemplos **nuevos** que nunca vio: **generalización**. Ejemplo: un auto autónomo debe manejar en calles donde nunca se entrenó.
2. **No supervisado:**
   - Hay ejemplos **sin etiquetas**.
   - El algoritmo descubre qué los distingue (los cubos tienen 6 caras; las esferas no tienen vértices, aunque cambie su tamaño o color) y los agrupa.
3. **Por refuerzo:**
   - La experiencia **no está disponible** de antemano: se consigue **interactuando**.
   - Se detalla en la siguiente diapositiva.

Diapositiva 17 — "Aprendizaje por refuerzo" (8 min)

Explica el ciclo **Agente ⇄ Ambiente** del diagrama:

1. El **agente** (el programa) hace una **acción**, por ejemplo mover una pieza de ajedrez.
2. La acción cambia el **estado** del **ambiente** (el tablero).
3. El agente **observa** el nuevo estado y recibe una **recompensa**.
4. Se repite muchas veces.

Conceptos:

- **Recompensa a corto plazo:** comer una pieza.
- **Recompensa a largo plazo (horizonte):** ganar la partida. Es lo que realmente importa.
- **Objetivo:** maximizar la recompensa total.
- Al inicio el agente pierde mucho. Con la experiencia ajusta su regla de "en este estado, haz esta acción" y termina ganando.

Cierre del tema: las redes neuronales sirven para muchas de estas tareas: regresión, clasificación, reducción de dimensionalidad y generación.

Diapositiva 18 — Quiz (10 min)

Que respondan en papel durante 5 minutos; luego revisa en voz alta.

#	Pregunta	Respuesta
1	¿Por qué el test de Turing es una definición operacional?	Solo mide si pasa o no; no importa cómo funciona por dentro.
2	¿En cuál de los tres ejes destacan las redes?	El operacional: resolver problemas.
3	¿Qué es generalizar?	Funcionar bien con ejemplos que nunca vio.
4	¿Diferencia entre clasificación y agrupamiento?	La clasificación tiene etiquetas predefinidas; el agrupamiento no.
5	¿Qué maximiza un agente de refuerzo?	La recompensa acumulada (a largo plazo).

Diapositiva 19 — "Para la próxima clase" (2 min)

1. Tener el ambiente instalado y el repositorio clonado (Parte A).
2. Leer **Russell y Norvig:** capítulo 1 y secciones 18.1 y 18.2 (*Learning from examples*).
3. Leer **Goodfellow, Bengio y Courville, *Deep Learning*:** sección 5.1 (*Learning algorithms*).

La próxima clase empezaremos con la primera neurona artificial: el modelo de **McCulloch y Pitts**.

PARTE C — Problemas frecuentes y soluciones

Síntoma	Causa	Solución
`conda` no se reconoce como comando	Abriste PowerShell o CMD normal	Abre **Anaconda PowerShell Prompt** desde el menú Inicio
Sigue diciendo `(base)`	No activaste el ambiente	`conda activate nni`
`EnvironmentNameNotFound: nni`	El ambiente no se creó	Repite el Paso 4 y revisa con `conda env list`
`git` no se reconoce	Git no instalado, o terminal abierta antes de instalarlo	Instala Git (Paso 2) y **abre una terminal nueva**
`mkdir` dice que la carpeta ya existe	Ya la habías creado	No es un error grave; sigue con `cd projects`
`destination path ... already exists`	Ya clonaste antes	No pasa nada; entra con `cd curso-redes-neuronales` y actualiza con `git pull`
`Could not open requirements file`	No estás dentro de `curso-redes-neuronales`	`cd ~\projects\curso-redes-neuronales` y repite el Paso 6
`ModuleNotFoundError: No module named 'numpy'` en VS Code	El kernel elegido no es `nni`	Select Kernel → Python Environments → **nni**

Síntoma	Causa	Solución
VS Code no muestra `nni` en la lista de kernels	Falta `ipykernel` o la extensión	Con `(nni)` activo: `pip install ipykernel`; instala las extensiones Python y Jupyter; reinicia VS Code
`ModuleNotFoundError: No module named 'nni'`	El notebook no se abrió desde el repositorio	Abre en VS Code la carpeta `curso-redes-neuronales` completa, no el archivo suelto
`pip` falla con error de red o SSL	Internet o proxy de la institución	Prueba otra red, o usa Colab
La instalación de Anaconda falla por permisos	Sin permisos de administrador	Usa Colab (Camino 1)
Las tildes se ven como `�` en la terminal	Codificación de Windows	Ejecuta `chcp 65001` antes; es solo visual
Colab perdió mis cambios	No se guardó una copia	**Archivo → Guardar una copia en Drive**

Si nada funciona: usa Colab en clase y revisa la instalación después, con calma, repitiendo la Parte A paso por paso.